

AnacostiaIQ Sensor REST API

Reference and command syntax

Generated 6 October 2026

Contents

1	Overview	1
1.1	Data model	1
1.2	How projects work	1
2	Endpoints	2
2.1	GET /health	2
2.2	POST /sensor	2
2.3	GET /sensor/<sensor_id>	3
2.4	GET /sensors	3
2.5	GET /projects	3
2.6	DELETE /sensor/<sensor_id>	3
3	JavaScript	4
3.1	Posting a reading	4
3.2	Reading a range	4
3.3	Listing sensors and projects	5
3.4	Deleting (token required)	5
4	Operations	5
4.1	Server access	5
4.2	Service control	5
4.3	Backing up the table	5
4.4	Acceptance tests	6
5	Client configuration	6
6	Known limitations	6

1 Overview

A Flask service stores sensor readings in Amazon DynamoDB and serves them over HTTP. Field stations POST readings; the dashboard and any other client GET them.

Base URL	http://54.213.147.59:5000
Host	ec2-54-213-147-59.us-west-2.compute.amazonaws.com
Service	sensor-api.service (systemd), ~/sensor-api/app.py
Storage	DynamoDB table sensor_readings, region us-west-2
Content type	application/json
CORS	enabled for all origins

1.1 Data model

Each reading is one item:

Field	Type	Meaning
<code>project</code>	string	Namespace. Groups one deployment's stations.
<code>sensor_id</code>	string	Sensor name, unique <i>within</i> a project.
<code>timestamp</code>	string	ISO 8601, YYYY-MM-DDTHH:MM:SS. Sort key.
<code>value</code>	string	Always stored and returned as a <i>string</i> .
<code>unit</code>	string	Free text, e.g. m, %, mm.

1.2 How projects work

The DynamoDB partition key is `sensor_id` and the sort key is `timestamp`, so a given pair identifies exactly one item. Without namespacing, two projects posting the same sensor name at the same instant would overwrite each other silently.

The API therefore stores the partition key as `<project>#<sensor_id>` and splits it apart again on read. Clients only ever see `project` and `sensor_id` as separate fields; the `#` encoding never appears on the wire.

- `project` must not contain `#`. A request that breaks this is rejected with 400.
- Omitting `project` reads and writes the *unnamedspace* space, which is what the API did before projects existed.
- `value` is stored by `str()`, so 42.5 comes back as "42.5". Cast on the client.
- A POST to an existing (`project`, `sensor_id`, `timestamp`) **overwrites** it. That is the only way to correct a reading — there is no PUT or PATCH.

2 Endpoints

Method	Path	Purpose
GET	<code>/health</code>	liveness check
POST	<code>/sensor</code>	store one reading
GET	<code>/sensor/<id></code>	read a sensor's readings
DELETE	<code>/sensor/<id></code>	delete readings (token required)
GET	<code>/sensors</code>	list sensor ids
GET	<code>/projects</code>	list project names

There is no authentication on reads or writes. Only DELETE is gated, by the X-Wipe-Token header.

2.1 GET /health

Returns `{"status":"ok"}` if the service is up.

```
curl http://54.213.147.59:5000/health
```

2.2 POST /sensor

Stores one reading. Body fields:

Field	Required	Default
sensor_id	yes	—
value	yes	—
project	no	none (unnamespaced)
unit	no	" "
timestamp	no	server UTC time

```
curl -X POST http://54.213.147.59:5000/sensor \
-H 'Content-Type: application/json' \
-d '{"project":"John-McCormack",
    "sensor_id":"lab_moisture_a0",
    "value":42.5,
    "unit":"%",
    "timestamp":"2026-10-06T19:00:00"}'
```

Letting the server supply the timestamp:

```
curl -X POST http://54.213.147.59:5000/sensor \
-H 'Content-Type: application/json' \
-d '{"project":"John-McCormack","sensor_id":"lab_moisture_a0","value":42.5,"unit":"%"}'
```

Responses: 200 {"status":"success"}; 400 if project contains #.

2.3 GET /sensor/<sensor_id>

Returns an array of readings, oldest first.

Query parameter	Meaning
project	restrict to one project; omit for unnamespaced data
start	inclusive lower bound on timestamp
end	inclusive upper bound on timestamp

start and **end** are compared as *strings* against the ISO timestamp, so a prefix such as 2026-10 works, but the format must match the stored one. Either may be given alone.

```
# everything for one sensor in one project
curl "http://54.213.147.59:5000/sensor/lab_moisture_a0?project=John-McCormack"

# a date range
curl "http://54.213.147.59:5000/sensor/lab_moisture_a0?project=John-McCormack&start=2026-10-01&
    ↪ end=2026-10-07"

# everything since a moment
curl "http://54.213.147.59:5000/sensor/lab_moisture_a0?project=John-McCormack&start=2026-10-06T00
    ↪ :00:00"

# pretty-printed
curl -s "http://54.213.147.59:5000/sensor/precip_amount?project=John-McCormack" | python3 -m json
    ↪ .tool
```

Example response:

```
[{"project":"John-McCormack","sensor_id":"lab_moisture_a0",
  "timestamp":"2026-10-06T19:00:00","unit":"%", "value":"42.5"}]
```

2.4 GET /sensors

Lists sensor ids. With `?project=` only that project's sensors; without it, every sensor in every project.

```
curl "http://54.213.147.59:5000/sensors?project=John-McCormack"
curl "http://54.213.147.59:5000/sensors"
```

This performs a full table scan, so it grows more expensive as the table does.

2.5 GET /projects

Lists distinct project names. A `null` entry means readings that carry no project.

```
curl http://54.213.147.59:5000/projects
```

2.6 DELETE /sensor/<sensor_id>

Destructive. Requires the `X-Wipe-Token` header. Omitting both `start` and `end` deletes *every* reading for that sensor in that project. Without a token configured server-side, deletion is refused outright.

Accepts the same `project`, `start` and `end` parameters as the `GET`, and returns the number removed.

```
# one sensor, one project, a bounded range
curl -X DELETE "http://54.213.147.59:5000/sensor/lab_moisture_a0?project=John-McCormack&start
↪ =2026-10-01&end=2026-10-02" \
-H "X-Wipe-Token: $WIPE_TOKEN"

# every reading for that sensor in that project
curl -X DELETE "http://54.213.147.59:5000/sensor/lab_moisture_a0?project=John-McCormack" \
-H "X-Wipe-Token: $WIPE_TOKEN"
```

Responses: 200 `{"status": "success", "sensor_id": ..., "deleted": N}`; 401 `{"error": "unauthorized"}`.

The token lives in the service environment on the server:

```
sudo systemctl show -p Environment --value sensor-api
```

3 JavaScript

All examples use `fetch`, and work in a browser or in Node 18+.

3.1 Posting a reading

```
const API = "http://54.213.147.59:5000";
const PROJECT = "John-McCormack";

async function postReading(sensorId, value, unit) {
  const res = await fetch(`${API}/sensor`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      project: PROJECT,
      sensor_id: sensorId,
      value: value,
      unit: unit,
    })
  });
}
```

```

        timestamp: new Date().toISOString().slice(0, 19) // no trailing Z
    })
});
if (!res.ok) throw new Error('POST failed: ${res.status}');
return res.json();
}

```

3.2 Reading a range

```

async function getReadings(sensorId, start, end) {
    const q = new URLSearchParams({ project: PROJECT });
    if (start) q.set("start", start);
    if (end) q.set("end", end);

    const res = await fetch(`${API}/sensor/${encodeURIComponent(sensorId)}?${q}`);
    if (!res.ok) throw new Error('GET failed: ${res.status}');

    const rows = await res.json();
    // value arrives as a string -- cast before plotting
    return rows.map(r => ({ t: new Date(r.timestamp), v: parseFloat(r.value) }));
}

const series = await getReadings("lab_moisture_a0",
    "2026-10-01", "2026-10-07");

```

3.3 Listing sensors and projects

```

const sensors = await (await fetch(`${API}/sensors?project=${PROJECT}`).json());
const projects = await (await fetch(`${API}/projects`).json());
// projects may contain null, meaning readings with no project

```

3.4 Deleting (token required)

```

async function deleteReadings(sensorId, token, start, end) {
    const q = new URLSearchParams({ project: PROJECT });
    if (start) q.set("start", start);
    if (end) q.set("end", end);

    const res = await fetch(`${API}/sensor/${encodeURIComponent(sensorId)}?${q}`, {
        method: "DELETE",
        headers: { "X-Wipe-Token": token }
    });
    if (res.status === 401) throw new Error("unauthorized -- bad or missing token");
    return res.json(); // { status, sensor_id, project, deleted }
}

```

4 Operations

4.1 Server access

```
ssh -i ~/Dropbox/AWS_/ArashLinux.pem ubuntu@54.213.147.59
```

4.2 Service control

```
systemctl status sensor-api
sudo systemctl restart sensor-api
journalctl -u sensor-api -f
```

4.3 Backing up the table

```
~/sensor-api/venv/bin/python3 - <<'PY'
import boto3, gzip, json, datetime
t = boto3.resource("dynamodb", region_name="us-west-2").Table("sensor_readings")
path = "backup-%s.json.gz" % datetime.datetime.now(datetime.UTC).strftime("%Y%m%d-%H%M%S")
kw, n = {}, 0
with gzip.open(path, "wt") as fh:
    while True:
        r = t.scan(**kw)
        for it in r["Items"]:
            fh.write(json.dumps({k: str(v) for k, v in it.items()}) + "\n"); n += 1
        if "LastEvaluatedKey" not in r: break
        kw["ExclusiveStartKey"] = r["LastEvaluatedKey"]
print(path, n, "items")
PY
```

4.4 Acceptance tests

```
python3 scripts/test_project_api.py
python3 scripts/test_project_api.py --token "$WIPE_TOKEN" # adds DELETE tests
```

5 Client configuration

The `project` value must agree on both sides, or the dashboard charts nothing.

Side	File	Key
Station (Pi)	<code>config.json</code>	"project" (top level)
Dashboard	<code>FrontEnd/config.json</code>	"project"

```
{
  "project": "John-McCormack",
  "station": { "id": "field_station_01", "name": "John McCormack Field Station" },
  "app": { "apiUrl": "http://54.213.147.59:5000/sensor" }
}
```

`project` groups stations; `station.id` still distinguishes them within it.

6 Known limitations

1. **No authentication on reads or writes.** Port 5000 is open to the world and only DELETE checks a token. Anyone who can reach the host can post arbitrary readings or read everything.
2. **No update endpoint.** Correcting a value means re-posting the same key, which overwrites.
3. **value is a string.** Clients must cast.

4. **/sensors and /projects scan the table.** Cost grows with table size; cache the result rather than polling.
5. **Timestamps are compared lexicographically.** Mixing formats (with and without a trailing Z) breaks range queries. Post YYYY-MM-DDTHH:MM:SS.
6. **Forecast and observation share the table.** Weather sensors such as `precip_amount` carry future timestamps.